

# Част I — Как да напишем `struct.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · от празен файл до готова програма

## Какво ще направим

Ще напишем програма, която пази име и пол на човек в **структура**. Данните ще се съхраняват в масиви от символи — така се работи в езика C, преди да съществуват класовете. Целта е да се види какво точно ни дава ООП, като първо построим варианта без него.

Ще пишем програмата на осем стъпки. **След всяка стъпка кодът се компилира**. Това е важно: ако сгрешите, ще знаете точно в коя стъпка, вместо да търсите грешката в 90 реда наведнъж.

### План на работата

1. Създаване на проекта във Visual Studio
2. Скелет на програмата
3. Структурата — само данните
4. Функции за промяна на данните (изискване 1)
5. Функции за четене на данните (изискване 2)
6. Функция за извеждане
7. Конструктори (изискване 3)
8. Главна функция (изискване 4)

## 1 Създаване на проекта

Във Visual Studio:

1. **File** → **New** → **Project**
2. Изберете **Empty Project** (в списъка с C++ шаблони). *Не* избирайте „Console App“ — той добавя допълнителни файлове, които само пречат в началото.
3. Дайте име на проекта, например `Tema1_Struct`, и натиснете **Create**.
4. В прозореца **Solution Explorer** щракнете с десен бутон върху **Source Files** → **Add** → **New Item**.
5. Изберете **C++ File (.cpp)**, напишете име `struct.cpp` и натиснете **Add**.

**Внимание** — една програма, една функция `main`. Ако по-късно добавите и `class.cpp` в *същия* проект, компилацията ще откаже с грешка `LNK2005`, защото двата файла съдържат по една функция `main`. Направете **отделен проект** за втората задача.

## 2 Скелет на програмата

НАПИШЕТЕ ТОВА В ПРАЗНИЯ ФАЙЛ

```
#include <stdio>      // getchar
#include <iostream>    // cout
using namespace std;

int main()
{
    cout << "Програмата работи!" << endl;

    getchar();        // задържа прозореца отворен
    return 0;
}
```

#### КАКВО ПРАВИ ВСЕКИ РЕД

| Ред                  | Обяснение                                                              |
|----------------------|------------------------------------------------------------------------|
| #include <iostream>  | Включва средствата за вход и изход. Без него cout не съществува.       |
| #include <stdio>     | Нужен е за getchar.                                                    |
| using namespace std; | Позволява да пишем cout вместо std::cout.                              |
| int main()           | Входната точка. Изпълнението на всяка програма започва оттук.          |
| getchar();           | Чака натискане на <b>Enter</b> . Вижте обяснението по-долу.            |
| return 0;            | Съобщава на операционната система, че програмата е приключила успешно. |

**Защо е нужен `getchar()` ?** Когато стартирате програмата с **F5** (Start Debugging), Visual Studio затваря конзолния прозорец веднага щом `main` завърши. Изходът се появява и изчезва за части от секундата. Редът `getchar()` кара програмата да чака натискане на **Enter**, преди да приключи — така успявате да прочетете резултата.

Алтернатива: стартирайте с **Ctrl + F5** (Start Without Debugging) — тогава Visual Studio сам задържа прозореца. Но при **F5** това не се случва, затова оставяме `getchar()` в кода.

**Проверка:** натиснете **Ctrl + Shift + B** (Build). В долния прозорец трябва да пише `1 succeeded, 0 failed`. Стартирайте с **F5** — прозорецът показва „Програмата работи!“ и чака.

### 3 Структурата — само данните

Сега описваме *какво* пази структурата. Функциите ще добавим след това.

#### ДОБАВЕТЕ НАД ФУНКЦИЯТА `MAIN`

```
struct CPerson
{
    char m_szName[64];    // име
    char m_szGender[32];  // пол
};
```

#### ОБЯСНЕНИЕ

**Защо масив от символи, а не `string` ?** Условието изисква точно това. Така се пази текст в езика C: поредица от символи, завършваща със специалния символ `'\0'` (нулев байт). Той отбелязва края на текста — без него програмата не знае докъде да чете.

**Защо 64 и 32?** В кодировката UTF-8 всяка кирилска буква заема **2 байта**, а не 1. Тоест `m_szName[64]` побира около **32 български букви**. Ако бяхте написали `[20]`, името „Александър Константинов“ нямаше да се побере.

**Защо имената започват с `m_sz`?** Това е приета конвенция: `m_` означава *member* (член на структурата), а `sz` — *string, zero-terminated* (низ, завършващ с нула). Не е задължително, но помага при четене на чужд код.

**Най-честата грешка тук:** пропусната **точка и запетая** след затварящата скоба. Пише се `};`, а не само `}`. Visual Studio дава подвеждащо съобщение `C2143: syntax error`, което сочи някъде другаде.

**Проверка:** компилирайте (`Ctrl` + `Shift` + `B`). Трябва да мине без грешки. Програмата още не използва структурата — това е нормално.

## 4 Функции за промяна на данните (изискване 1)

Първо да видим **грешния** начин, за да е ясно защо правилният изглежда сложно:

```
// НЕ пишете това!
void SetName(const char* sz)
{
    strcpy(m_szName, sz);    // опасно
}
```

Функцията `strcpy` копира текста, без да проверява дали има място. Ако някой подаде име от 200 символа, тя ще запише 200 байта в буфер от 64 — и ще повреди паметта след него. Това се нарича **препълване на буфера** и е една от най-честите причини за пробиви в сигурността.

### НАПИШЕТЕ ТОВА ВЪТРЕ В СТРУКТУРАТА, СЛЕД ДВЕТЕ ПРОМЕНЛИВИ

```
void SetName(const char* sz)
{
    if (!sz) sz = "";           // 1
    strncpy(m_szName, sz, sizeof(m_szName) - 1); // 2
    m_szName[sizeof(m_szName) - 1] = '\0';      // 3
}
```

### ТРИТЕ РЕДА, ЕДИН ПО ЕДИН

| № | Какво прави и защо                                                                                                                                                                                                                                 |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Ако някой подаде <code>nullptr</code> , заменяме го с празен текст. Без тази проверка <code>strncpy</code> с нулев указател предизвиква срив на програмата.                                                                                        |
| 2 | Копира най-много 63 байта. Записваме <code>sizeof(m_szName) - 1</code> , а не 63, за да остане вярно и ако утре смените размера на масива. Изваждаме едно, за да запазим място за нулата.                                                          |
| 3 | Поставя нулата ръчно. <b>Това е най-важният ред.</b> Когато текстът запълни целия буфер, <code>strncpy</code> <i>не слага</i> завършваща нула — за разлика от това, което подсказва името ѝ. Без този ред програмата ще чете отвъд края на масива. |

**Сега вие:** напишете по същия образец функцията `SetGender`. Тя е абсолютно същата — само заменете `m_szName` с `m_szGender` на трите места. Внимавайте да смените и в `sizeof`.

### ДОБАВЕТЕ НАЙ-ГОРЕ ВЪВ ФАЙЛА, ПРЕДИ ВСИЧКИ `#INCLUDE`

```
#define _CRT_SECURE_NO_WARNINGS
#include <cstring> // strcpy
```

**Защо е нужен този #define ?** Visual Studio смята `strcpy` за остаряла и по подразбиране **отказва** да компилира с грешка C4996, предлагайки своята версия `strcpy_s`. Редът `_CRT_SECURE_NO_WARNINGS` изключва това ограничение. **Задължително** трябва да стои *преди* всички `#include` — ако е след тях, няма ефект.

**Проверка:** компилирайте. Ако получите C4996, значи `#define` редът е на грешно място — преместете го най-отгоре.

## 5 Функции за четене на данните (изискване 2)

### ДОБАВЕТЕ ВЪТРЕ В СТРУКТУРАТА

```
const char* GetName() const { return m_szName; }
const char* GetGender() const { return m_szGender; }
```

### ДВЕТЕ ДУМИ `CONST` ОЗНАЧАВАТ РАЗЛИЧНИ НЕЩА

| Къде стои                             | Какво означава                                                                                                                                                           |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>const char*</code> — в началото | „Връщам ви указател към текста, но <b>нямате право да го промените</b> през него.“ Защи́тава вътрешните данни от промяна отвън, заобикаляйки <code>SetName</code> .      |
| <code>const</code> — след скобите     | „Тази функция <b>не променя обекта</b> .“ Обещание към компилатора. Благодарение на него функцията може да се вика и върху константен обект — ще го проверим в стъпка 8. |

**Проверка:** компилирайте отново.

## 6 Функция за извеждане

За да не пишем едно и също при всяко отпечатване, добавяме помощна функция.

### ДОБАВЕТЕ ВЪТРЕ В СТРУКТУРАТА

```
void Print() const
{
    cout << GetName() << " (" << GetGender() << ")" << endl;
}
```

Обърнете внимание: използваме `GetName()`, а не направо `m_szName`. Резултатът е същият, но така, ако утре променим начина на съхранение, ще трябва да поправим само една функция.

## 7 Конструктори (изискване 3)

Първо вижте **проблема**. Добавете временно в `main`:

```
CPerson test;
test.Print();
```

Резултатът е случайни символи или срыв. Причината: обявявайки `CPerson test;`, ние заделяме 96 байта памет, но **не записваме нищо в тях**. Там стои каквото е останало от предишни програми.

Конструкторът е специална функция, която се изпълнява **автоматично** при създаване на обекта. Име — същото като на структурата, без тип на връщаната стойност.

#### ДОБАВЕТЕ ВЪТРЕ В СТРУКТУРАТА, НАД ФУНКЦИИТЕ SET...

```
// Подразбиращ се конструктор – вика се при CPerson person;
CPerson() { SetName("Неизвестен"); SetGender("неопределен"); }

// Експлицитен конструктор – вика се при CPerson person("Иван", "мъж");
CPerson(const char* szName, const char* szGender)
{ SetName(szName); SetGender(szGender); }
```

**Изглежда като грешка, но не е.** Конструкторът вика `SetName`, а тя е написана *по-долу* във файла. В обикновен код това би било грешка. Вътре в структура обаче компилаторът първо прочита цялото тяло и чак тогава проверява функциите — затова редът на дефиниране няма значение.

**Проверка:** компилирайте и стартирайте. Сега `test.Print()` извежда Неизвестен (неопределен) вместо боклук. Изтрийте временните два реда.

## 8 Главна функция (изискване 4)

Изискването е да създадем променливи „от различни типове“. Ще добавяме по една група наведнъж и ще компилираме след всяка.

### 8.1 Обикновени обекти — двата конструктора

```
CPerson person1;                // подразбиращ се конструктор
CPerson person2("Иван Петров", "мъж"); // експлицитен конструктор

cout << "---- Обикновени обекти ----" << endl;
person1.Print();
person2.Print();
```

### 8.2 Промяна чрез функциите за запис

```
cout << "---- След промяна ----" << endl;
person1.SetName("Мария Иванова");
person1.SetGender("жена");
person1.Print();
```

### 8.3 Масив от обекти

```
CPerson people[3] = {
    CPerson("Георги Стоянов", "мъж"),
    CPerson("Елена Димитрова", "жена"),
    CPerson() // подразбиращите се стойности
};

cout << "---- Масив от обекти ----" << endl;
const int nCount = sizeof(people) / sizeof(people[0]);
for (int i = 0; i < nCount; i++)
    people[i].Print();
```

Изразът `sizeof(people) / sizeof(people[0])` дава броя на елементите: общият размер, разделен на размера на един елемент. Ако добавите четвърти човек, броят се преизчислява сам.

### 8.4 Указател и динамична памет

```
cout << "---- Указател (динамична памет) ----" << endl;
CPerson* pPerson = new CPerson("Петър Колев", "мъж");
pPerson->Print(); // стрелка вместо точка
delete pPerson;   // освобождаване на паметта
pPerson = nullptr;
```

При обект, създаден с `new`, достъпът до функциите става със **стрелка** `->`, а не с точка. Всяко `new` задължително се вдвоява с `delete` — иначе паметта остава заета до края на програмата. Присвояването на `nullptr` след това предпазва от случайно използване на вече освободен указател.

## 8.5 Референция

```
cout << "---- Референция ----" << endl;
CPerson& refPerson = person2;
refPerson.SetGender("неопределен");      // променя самия person2
person2.Print();
```

Референцията **не е копие**, а второ име на същия обект. Затова промяната през `refPerson` се вижда при отпечатване на `person2`. Обърнете внимание на изхода — това е основната разлика спрямо копие.

## 8.6 Константен обект

```
cout << "---- Константен обект ----" << endl;
const CPerson cPerson("Анна Тодорова", "жена");
cout << cPerson.GetName() << " / " << cPerson.GetGender() << endl;
// cPerson.SetName("Друго");           // грешка: обектът е константен
```

**Задача за проверка:** разкомментирайте последния ред и компилирайте. Visual Studio трябва да **откаже** с грешка C2662 — не може да се вика функция, променяща обекта, върху константен обект. Това е *очакваният* резултат и точно заради него функциите за четене са маркирани с `const`. Върнете реда като коментар.

## 8.7 Задържане на прозореца

```
getchar();
return 0;
```

## Компилиране и стартиране

### Вариант А — през средата на Visual Studio

| Клавиши          | Действие                                                                                                                              |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Ctrl + Shift + B | <b>Build</b> — само компилира. Резултатът се вижда в прозореца Output.                                                                |
| F5               | <b>Start Debugging</b> — компилира и стартира с дебъгер. Прозорецът се затваря веднага след main, затова е нужен getchar().           |
| Ctrl + F5        | <b>Start Without Debugging</b> — Visual Studio сам задържа прозореца с надпис „Press any key to continue“.                            |
| F9               | Поставя <b>точка на прекъсване</b> на текущия ред. При F5 изпълнението спира там и може да се разглеждат стойностите на променливите. |

### Вариант Б — от команден ред

Компиляторът `cl` не работи в обикновен `cmd`. Трябва да отворите **Developer Command Prompt for VS** (търси се в старт менюто). Той настройва пътищата към компилатора.

```
cl /EHsc /std:c++17 /utf-8 struct.cpp
struct.exe
```

| Флаг       | Значение                                                                                         |
|------------|--------------------------------------------------------------------------------------------------|
| cl         | Компиляторът на Microsoft.                                                                       |
| /EHsc      | Включва обработката на изключения. Без него компилаторът дава предупреждение C4530.              |
| /std:c++17 | Задава стандарта на езика.                                                                       |
| /utf-8     | Указва, че файлът е записан в UTF-8. <b>Задължителен при кирилица</b> — вижте следващата секция. |
| struct.cpp | Исходният файл. Резултатът се казва struct.exe.                                                  |

Флаговете на Microsoft започват с наклонена черта / , а не с тире. Ако сте виждали команди с `g++ -std=c++17`, това е другият компилатор (GCC), използван в Linux и macOS. Кодът е един и същ — различават се само командите.

## Кирилица под Windows

Ако вместо български букви виждате ??? или безсмислени символи, причината е една от три. Проверете ги в този ред:

### 1. Файлът трябва да е записан в UTF-8

Във Visual Studio: **File** → **Save As**, до бутона **Save** има малка стрелка → **Save with Encoding** → изберете **Unicode (UTF-8 with signature)** — **Codepage 65001**.

Думата „signature“ означава невидим маркер в началото на файла, по който Visual Studio разпознава кодировката автоматично.

## 2. Компиляторът трябва да знае за UTF-8

При команден ред добавете флага `/utf-8`. В средата: **Project** → **Properties** → **C/C++** → **Command Line** и в полето **Additional Options** напишете `/utf-8`.

## 3. Конзолата трябва да е в UTF-8

Конзолата на Windows по подразбиране използва стара кодировка. Има два начина да се смени:

**Ръчно** — преди стартиране на програмата изпълнете в същия прозорец:

```
chcp 65001
```

**От самата програма** — добавете като първи ред в `main`:

```
#include <cstdlib>    // горе при останалите include-ове

int main()
{
    system("chcp 65001 > nul");    // кирилица в конзолата
    ...
}
```

Вторият вариант работи винаги, без да се помни нищо. Недостатъкът му е, че `system` стартира външна програма — при истински проект това се избягва, но за учебна задача е напълно приемливо.

**Шрифтът също има значение.** Ако след `chcp 65001` все още виждате квадратчета, щракнете с десен бутон върху заглавната лента на конзолата → **Properties** → **Font** и изберете **Consolas** или **Lucida Console**. Растерният шрифт по подразбиране не съдържа кирилица.

## Пълен код на програмата

---

Ако някъде сте се загубили, сравнете с този листинг:

```

// Тема 1: Въведение в ООП. Структури на C срещу класове на C++.

// Visual Studio отказва strncpy без този ред. Трябва да е ПРЕДИ include-овете.
#define _CRT_SECURE_NO_WARNINGS

#include <cstdio>      // getchar
#include <cstring>     // strncpy
#include <iostream>
using namespace std;

// ===== I. Структура CPerson (масиви от символи, стил C) =====
struct CPerson
{
    // Членовете на struct са публични по подразбиране.
    // 64 байта = 32 букви: кирилицата в UTF-8 е по 2 байта на буква.
    char m_szName[64];
    char m_szGender[32];

    // 3. Подразбиращ се конструктор
    CPerson() { SetName("Неизвестен"); SetGender("неопределен"); }

    // 3. Експлицитен конструктор (с параметри)
    CPerson(const char* szName, const char* szGender)
    { SetName(szName); SetGender(szGender); }

    // 1. Функции за запис; strncpy + ръчна нула => няма препълване
    void SetName(const char* sz)
    {
        if (!sz) sz = "";
        strncpy(m_szName, sz, sizeof(m_szName) - 1);
        m_szName[sizeof(m_szName) - 1] = '\0';
    }
    void SetGender(const char* sz)
    {
        if (!sz) sz = "";
        strncpy(m_szGender, sz, sizeof(m_szGender) - 1);
        m_szGender[sizeof(m_szGender) - 1] = '\0';
    }

    // 2. Функции за четене; const => не променят обекта
    const char* GetName() const { return m_szName; }
    const char* GetGender() const { return m_szGender; }

    void Print() const
    {
        cout << GetName() << " (" << GetGender() << ")" << endl;
    }
};

// ===== 4. Главна функция: променливи от различни типове =====
int main()
{
    // а) обикновени обекти – двата конструктора
    CPerson person1;           // подразбиращ се конструктор
    CPerson person2("Иван Петров", "мъж"); // експлицитен конструктор

    cout << "---- Обикновени обекти ----" << endl;
    person1.Print();
    person2.Print();
}

```

```

// б) промяна на данните чрез функциите за запис
cout << "---- След промяна ----" << endl;
person1.SetName("Мария Иванова");
person1.SetGender("жена");
person1.Print();

// в) масив от обекти
CPerson people[3] = {
    CPerson("Георги Стоянов", "мъж"),
    CPerson("Елена Димитрова", "жена"),
    CPerson() // подразбиращите се стойности
};

cout << "---- Масив от обекти ----" << endl;
const int nCount = sizeof(people) / sizeof(people[0]);
for (int i = 0; i < nCount; i++)
    people[i].Print();

// г) указател към обект в динамичната памет
cout << "---- Указател (динамична памет) ----" << endl;
CPerson* pPerson = new CPerson("Петър Колев", "мъж");
pPerson->Print(); // достъп чрез ->
delete pPerson; // освобождаване на паметта
pPerson = nullptr;

// д) референция – псевдоним на съществуващ обект
cout << "---- Референция ----" << endl;
CPerson& refPerson = person2;
refPerson.SetGender("неопределен"); // променя самия person2
person2.Print();

// е) константен обект – позволени са само const функциите за четене
cout << "---- Константен обект ----" << endl;
const CPerson cPerson("Анна Тодорова", "жена");
cout << cPerson.GetName() << " / " << cPerson.GetGender() << endl;
// cPerson.SetName("Друго"); // грешка: обектът е константен

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

## Очакван изход

```
---- Обикновени обекти ----
Неизвестен (неопределен)
Иван Петров (мъж)
---- След промяна ----
Мария Иванова (жена)
---- Масив от обекти ----
Георги Стоянов (мъж)
Елена Димитрова (жена)
Неизвестен (неопределен)
---- Указател (динамична памет) ----
Петър Колев (мъж)
---- Референция ----
Иван Петров (неопределен)
---- Константен обект ----
Анна Тодорова / жена
```

След последния ред програмата чака натискане на `Enter` заради `getchar()`.

## Чести грешки

| Код     | Съобщение                                                         | Причина и решение                                                                                                                         |
|---------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| C4996   | 'strncpy': This function or variable may be unsafe                | Липсва <code>#define _CRT_SECURE_NO_WARNINGS</code> или е поставен след някой <code>#include</code> . Преместете го най-отгоре във файла. |
| C2143   | syntax error: missing ';' before '}'                              | Пропуснатата точка и запетая след затварящата скоба на структурата. Пише се <code>};</code>                                               |
| C2065   | 'cout': undeclared identifier                                     | Липсва <code>#include &lt;iostream&gt;</code> или <code>using namespace std;</code>                                                       |
| C3861   | 'getchar': identifier not found                                   | Липсва <code>#include &lt;cstdio&gt;</code>                                                                                               |
| C2662   | cannot convert 'this' pointer from 'const CPerson' to 'CPerson &' | Извиквате функция за запис върху константен обект. Ако е тестът от 8.6 — това е очакваното поведение.                                     |
| LNK2005 | main already defined                                              | В проекта има два файла с функция <code>main</code> . Направете отделен проект за всяка задача.                                           |
| LNK1120 | unresolved externals                                              | Липсва функция <code>main</code> или е сгрешено името ѝ.                                                                                  |

## Известно ограничение

Функциите `SetName` и `SetGender` отрязват текста по **байтове**, а не по букви. Понеже кирилската буква е 2 байта, отрязването може да среже буква наполовина и на екрана да се появи `0`. Буферът остава защитен от препълване — гаранцията е „няма препълване“, а не „винаги валиден текст“.

Това не е грешка в решението, а естествено ограничение на подхода с масиви от символи. В част II ще видите как STL типът `string` премахва целия този клас проблеми.